



ENS2001

Ensamblador y Simulador del Lenguaje Ensamblador *IEEE 694*

Autor: Federico J. Álvarez Valero

Tutor: José Luis Fuertes Castro

Facultad de Informática – Marzo 2003

ÍNDICE



- Introducción
- Estado de la Cuestión
- Solución
 - Especificación de Requisitos
 - Ensamblador
 - Simulador
 - Interfaces
 - Pruebas
- Conclusiones
- Futuras Líneas de Trabajo

INTRODUCCIÓN



- **Introducción**
- Estado de la Cuestión
- Solución
 - Especificación de Requisitos
 - Ensamblador
 - Simulador
 - Interfaces
 - Pruebas
- Conclusiones
- Futuras Líneas de Trabajo

INTRODUCCIÓN

¿Qué es *ENS2001*?

- Simulador
de un sistema basado en arquitectura *Von Neumann*.
- Ensamblador
del lenguaje ensamblador *IEEE 694*.

INTRODUCCIÓN

Construcción de un simulador.

- Motivación.
 - Modelizar un sistema en el que los alumnos de Compiladores puedan probar sus prácticas.
- Características.
 - Simular un sistema de arquitectura sencilla.
 - Mejorar el aspecto y productividad de las herramientas actuales.

INTRODUCCIÓN

Construcción de un ensamblador.

- Motivación.
 - Definir un lenguaje ensamblador que se ejecute sobre la arquitectura simulada.
- Características.
 - Lenguaje ensamblador estándar.
 - Traduce los programas generados por las prácticas de los alumnos en código máquina que será ejecutado por el simulador

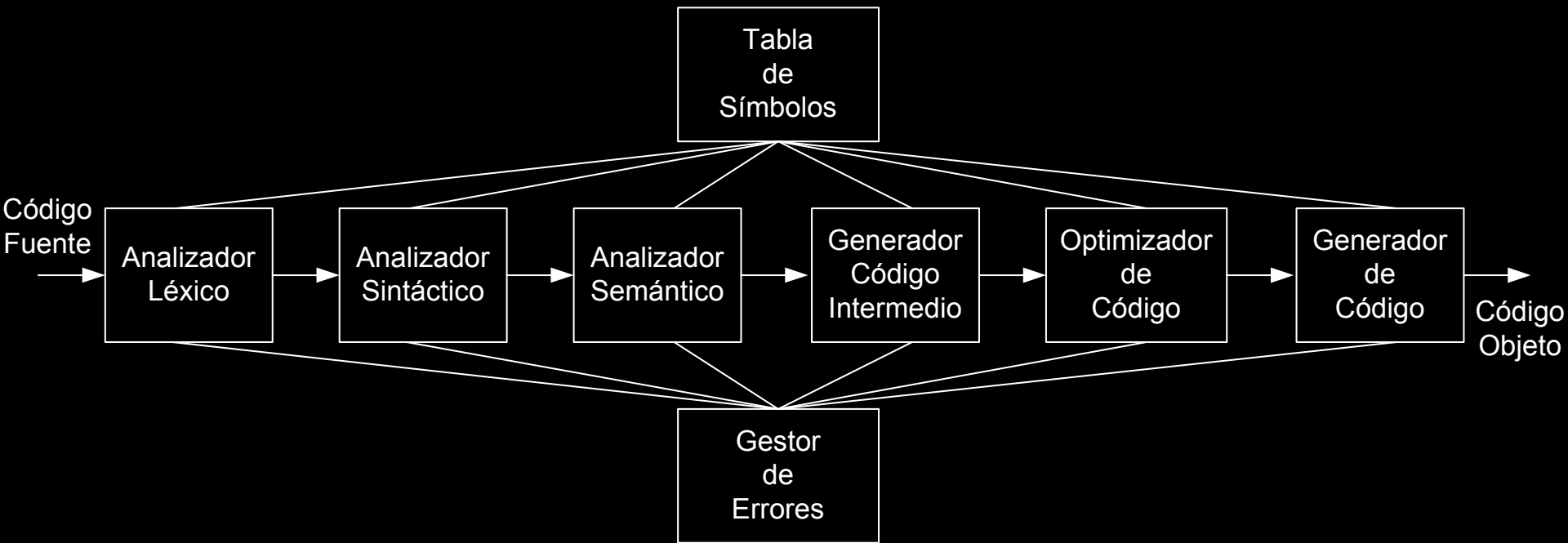
ESTADO DE LA CUESTIÓN



- Introducción
- Estado de la Cuestión
- Solución
 - Especificación de Requisitos
 - Ensamblador
 - Simulador
 - Interfaces
 - Pruebas
- Conclusiones
- Futuras Líneas de Trabajo

ESTADO DE LA CUESTIÓN

Teoría de compiladores.



ESTADO DE LA CUESTIÓN

Predecesores de *ENS2001*. *ASS (1992)* y *ENS96 (1996)*.

Ventajas.

- *ENS96* adopta el estándar *IEEE 694*.
- *ENS96* permite fácilmente el seguimiento de la simulación.
- Ambas tienen depurador e interfaz de comandos.
- *ENS96* permite generar código reubicable.

Desventajas.

- Faltan algunos biestables de estado (acarreo, paridad).
- El juego de instrucciones no es homogéneo.
- No existe el modo de direccionamiento relativo a PC.
- El formato de instrucción es único.
- No existe una instrucción que lea cadenas.
- *ENS96* sólo permite un punto de ruptura.

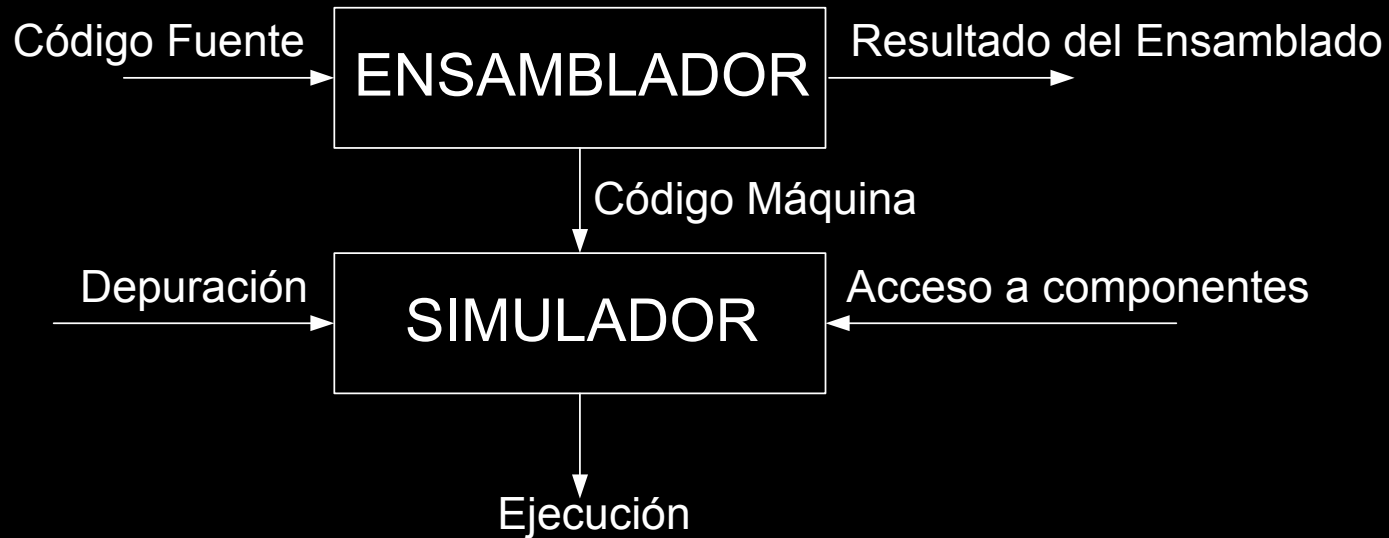
ESPECIFICACIÓN DE REQUISITOS



- Introducción
- Estado de la Cuestión
- **Solución**
 - Especificación de Requisitos
 - Ensamblador
 - Simulador
 - Interfaces
 - Pruebas
- Conclusiones
- Futuras Líneas de Trabajo

ESPECIFICACIÓN DE REQUISITOS

Funcionalidad de la aplicación.

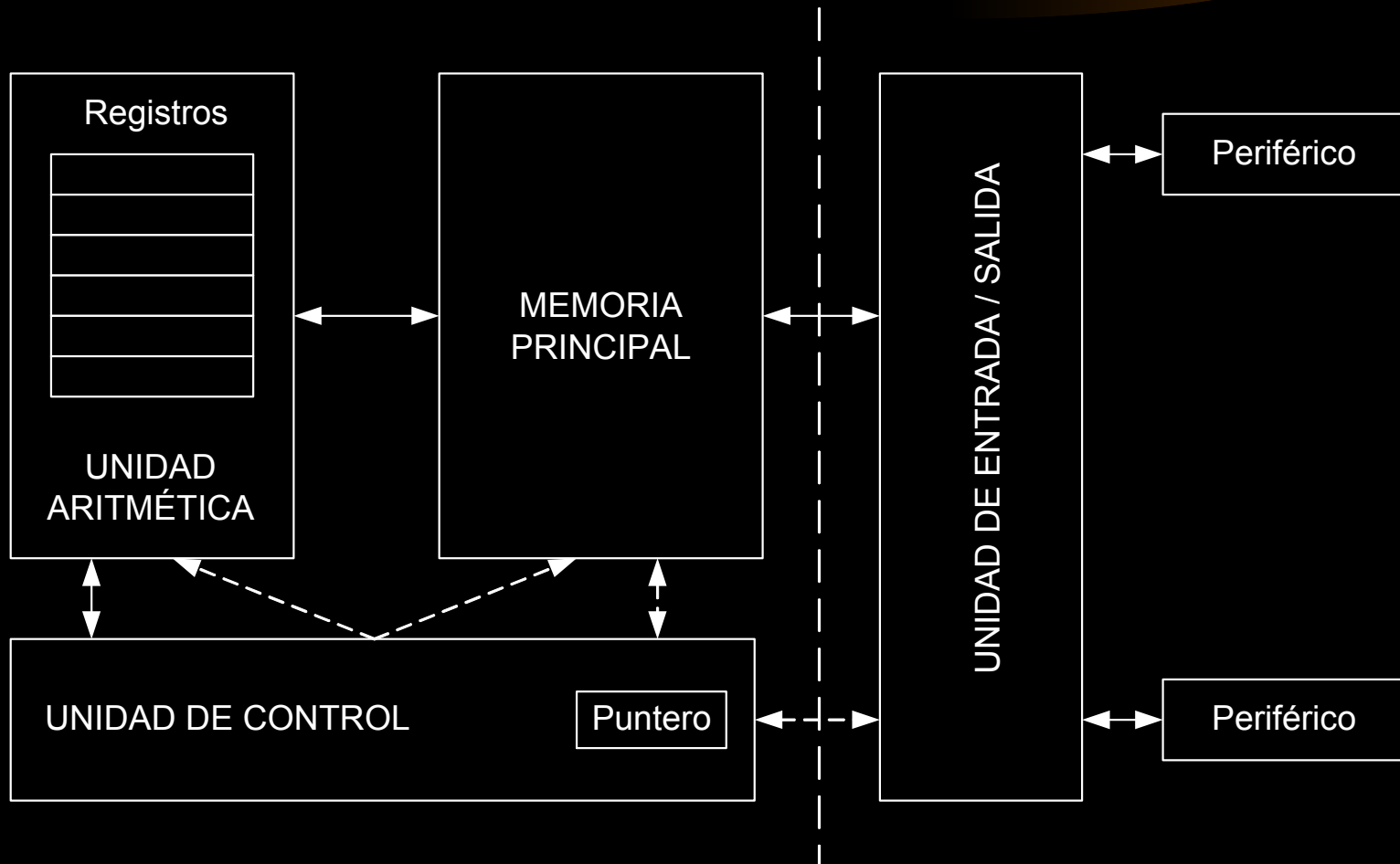


Versiones.

- *DOS.*
- *Linux.*
- *Windows.*

ESPECIFICACIÓN DE REQUISITOS

Arquitectura *Von Neumann*.



ESPECIFICACIÓN DE REQUISITOS

Lenguaje Ensamblador Estándar *IEEE 694*.

- Publicado en 1985. Revisado en 1994.
- Define un lenguaje ensamblador común a diferentes arquitecturas de microprocesador.
- Es un estándar explicado a los alumnos a lo largo de la carrera de Informática.

- Introducción
- Estado de la Cuestión
- **Solución**
 - Especificación de Requisitos
 - **Ensamblador**
 - Simulador
 - Interfaces
 - Pruebas
- Conclusiones
- Futuras Líneas de Trabajo

Estructura del código fuente.

```
[ [Etiqueta ':' ] Instrucción] [';' Comentario]
```

```
Instrucción := Mnemónico [Op1 [',' Op2] ]
```

ENSAMBLADOR

Juego de instrucciones

Transferencia de datos

Aritméticas

Comparación

Lógicas

Bifurcación

Manejo de Subrutinas

Entrada/Salida

Otras

Modos de direccionamiento

Inmediato

Directo a registro

Directo a memoria

Relativo a PC

Indirecto

Relativo a registro índice

ENSAMBLADOR

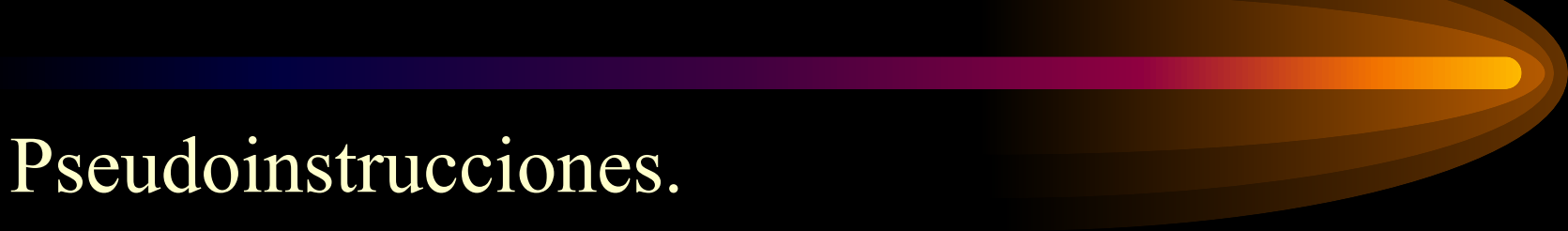
Formatos de instrucción.

- Se han definido formatos de 2, 4 y 6 bytes.
- La longitud media de instrucción es de 4,6 bytes.
- Se consigue ahorrar un 19 % de espacio.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Código de operación								M.dir op1				M.dir op2			

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Operando 1								Operando 2							

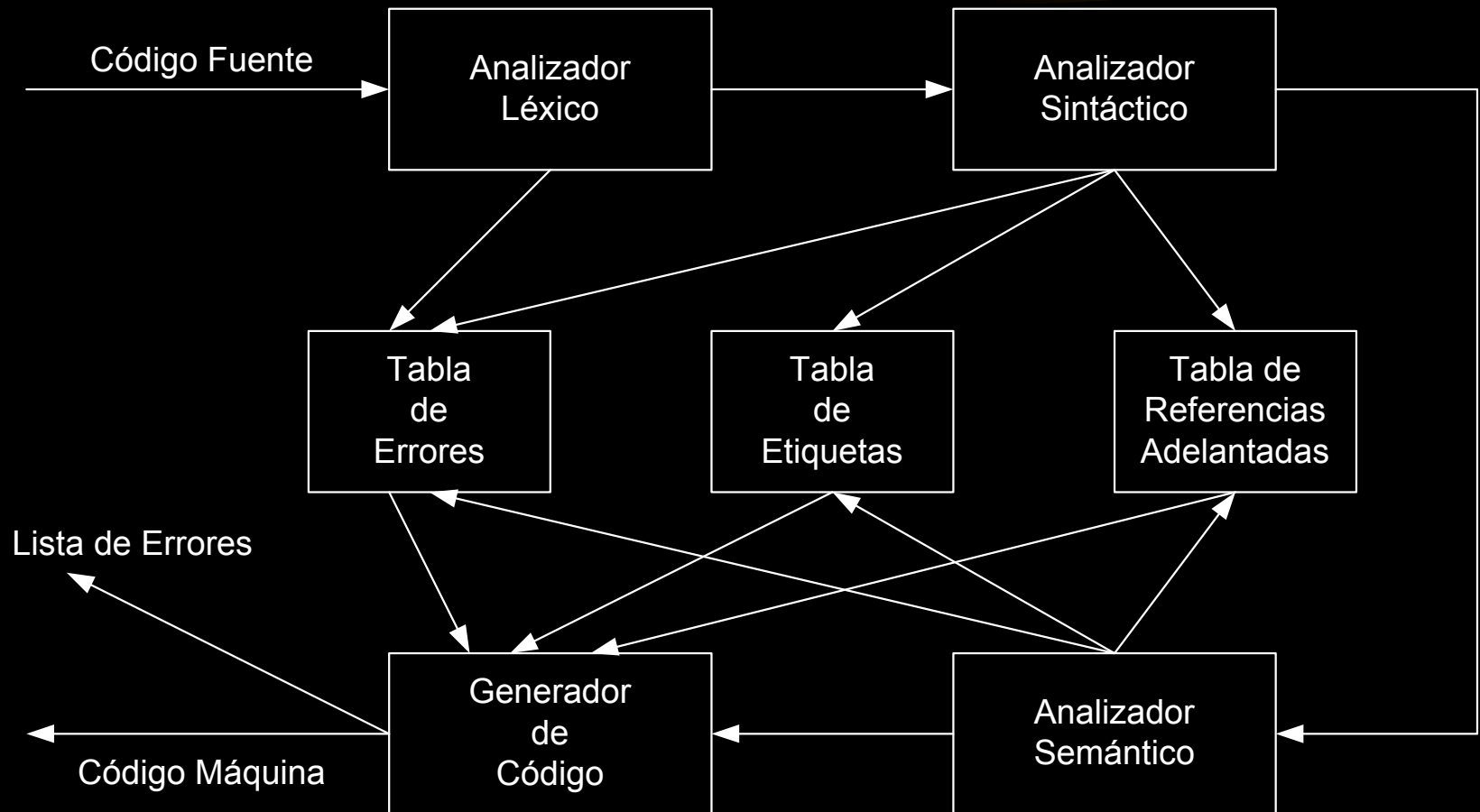
ENSAMBLADOR



Pseudoinstrucciones.

- **ORG**
- **END**
- **EQU**
- **RES**
- **DATA**

Esquema modular del ensamblador.



Analizador Léxico.

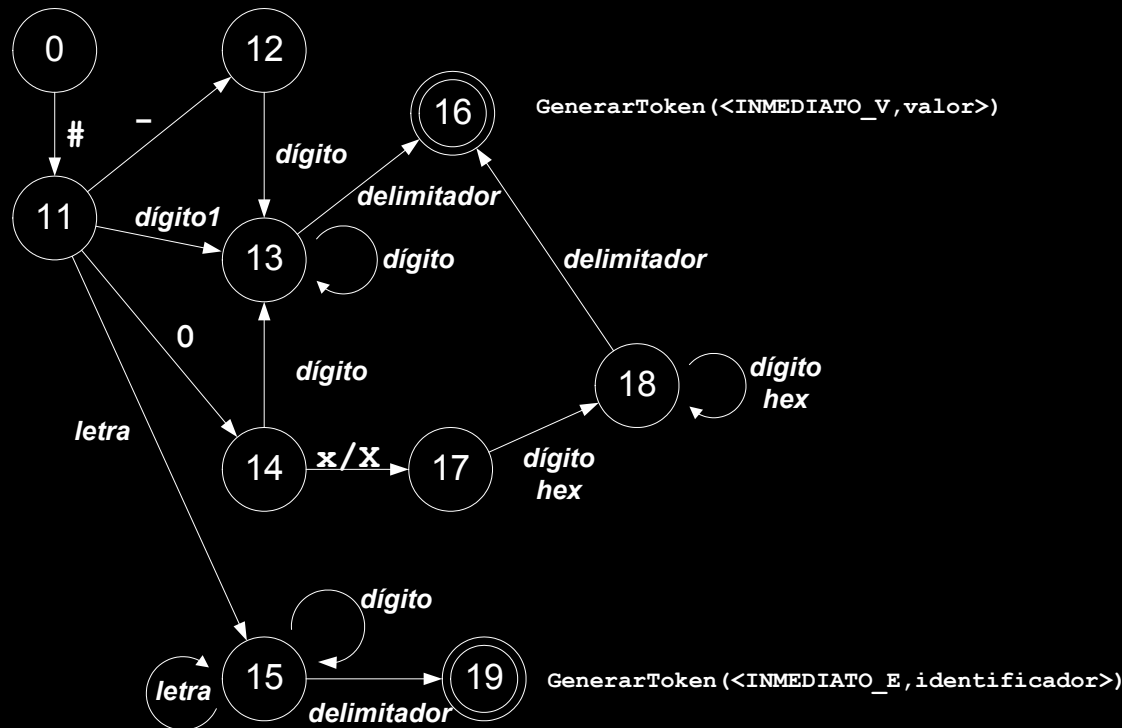
- Identificar *tokens*.
- Gramática del analizador léxico (fragmento).

$$S \rightarrow \text{'\#'} A \mid \dots$$
$$A \rightarrow d_1 A_1 \mid \text{'0'} A_2 \mid \text{'-'} A_3 \mid l A_4$$
$$A_1 \rightarrow d A_1 \mid \lambda$$
$$A_2 \rightarrow \text{'x'} A_5 \mid \text{'X'} A_5 \mid d A_1 \mid \lambda$$
$$A_3 \rightarrow d A_1 \mid d$$
$$A_4 \rightarrow l A_4 \mid d A_4 \mid \lambda$$
$$A_5 \rightarrow d_h A_6$$
$$A_6 \rightarrow d_h A_6 \mid \lambda$$

ENSAMBLADOR

Analizador Léxico.

- Autómata Finito Determinista.
- Acciones Semánticas.



Analizador Sintáctico.

- Fragmento de la gramática.

```
EXPRESION → EXPRESIÓN <SUMA> EXPRESIÓN2
|
|           EXPRESIÓN <RESTA> EXPRESIÓN2
|           EXPRESIÓN2
EXPRESIÓN2 → EXPRESIÓN2 <PRODUCTO> EXPRESIÓN3
|
|           EXPRESIÓN2 <DIVISIÓN> EXPRESIÓN3
|           EXPRESIÓN2 <MÓDULO> EXPRESIÓN3
|           EXPRESIÓN3
EXPRESIÓN3 → <PARENT_ABRE> EXPRESIÓN <PARENT_CIERRA>
|
|           <RESTA> EXPRESIÓN
|           <ENTERO>
```

ENSAMBLADOR

Analizador semántico. Se diseña una DDS.

<i>EXPRESIÓN2</i> → <i>EXPRESIÓN2₁</i> <PRODUCTO> <i>EXPRESIÓN3</i>	<code>EXPRESIÓN2.valor := EXPRESIÓN2₁.valor * EXPRESIÓN3.valor;</code>
<i>EXPRESIÓN2</i> → <i>EXPRESIÓN2₁</i> <DIVISIÓN> <i>EXPRESIÓN3</i>	<code>SI (EXPRESIÓN3.valor == 0) ENTONCES EnviarError(EXPR_DIVISIÓN_POR_CERO,nlin,0); SI NO EXPRESIÓN2.valor := EXPRESIÓN2₁.valor / EXPRESIÓN3.valor;</code>
<i>EXPRESIÓN2</i> → <i>EXPRESIÓN2₁</i> <MÓDULO> <i>EXPRESIÓN3</i>	<code>SI (EXPRESIÓN3.valor == 0) ENTONCES EnviarError(EXPR_DIVISIÓN_POR_CERO,nlin,0); SI NO EXPRESIÓN2.valor := EXPRESIÓN2₁.valor % EXPRESIÓN3.valor;</code>
<i>EXPRESIÓN2</i> → <i>EXPRESIÓN3</i>	<code>EXPRESIÓN2.valor := EXPRESIÓN3.valor;</code>

ENSAMBLADOR

Estructuras internas.

Tabla de etiquetas

Identificador	Valor
Etiqueta_1	33
Etiqueta_2	¿?
Bucle	10

Tabla de referencias adelantadas

Identificador	Dirección	Desplazamiento	Tamaño
Etiqueta_1	20	0	16 bits
Etiqueta_2	24	8	8 bits
Etiqueta_3	26	0	8 bits

Tabla de errores

Número	Descripción	Token	Nº Línea
1	Modo Direccionamiento Erróneo	#31[.ix]	4
2	Se esperaba Fin de Línea	.r3	12

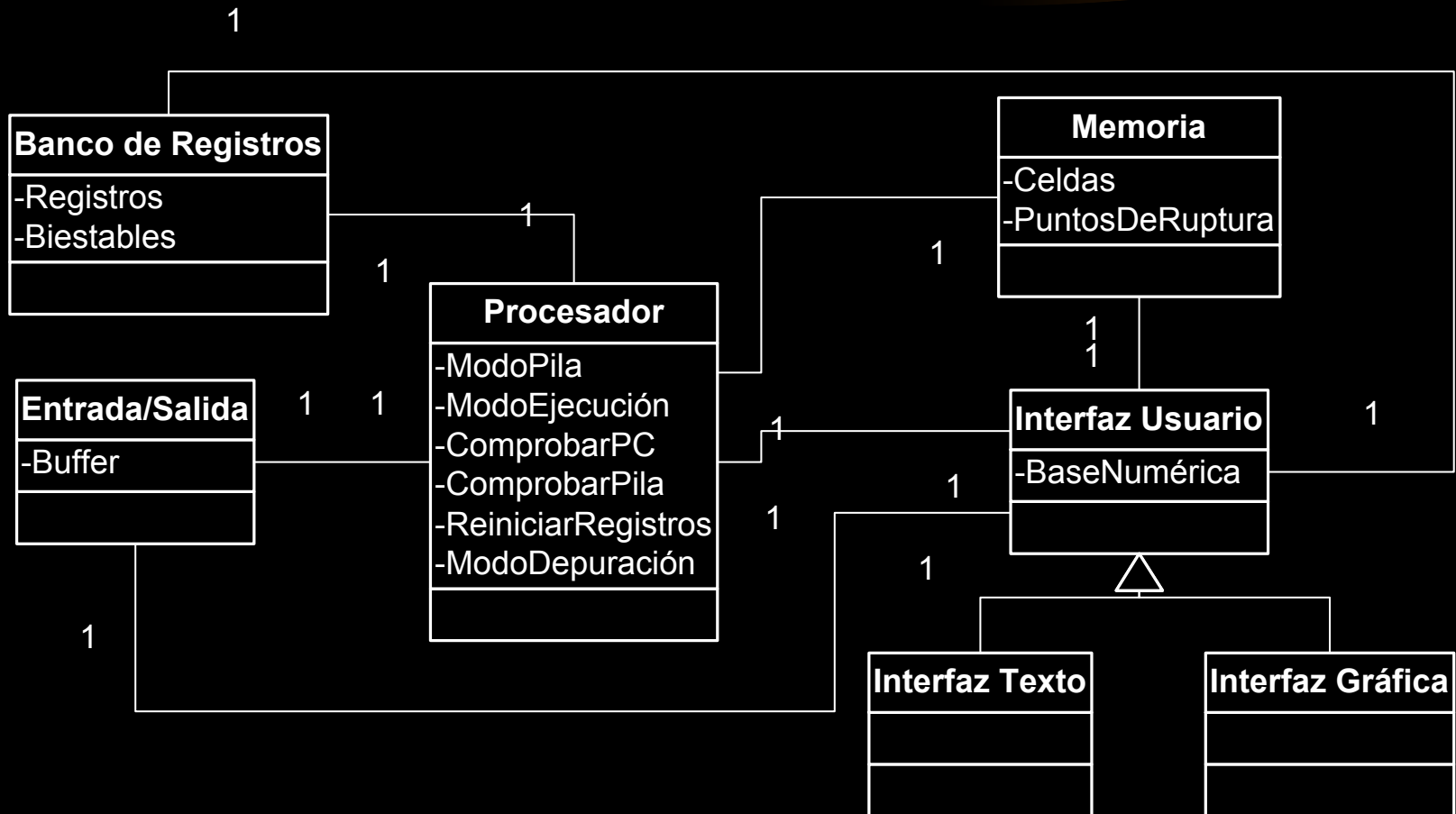
ENSAMBLADOR

Implementación.

- Uso de herramientas de generación automática (*Flex y Bison*).
- Código escrito en lenguaje C.
- Herramientas de desarrollo:
 - *Borland C++ Builder 5 (DOS y Windows)*.
 - *GNU Compiler Collection - gcc v.2.95.4 (Linux)*.

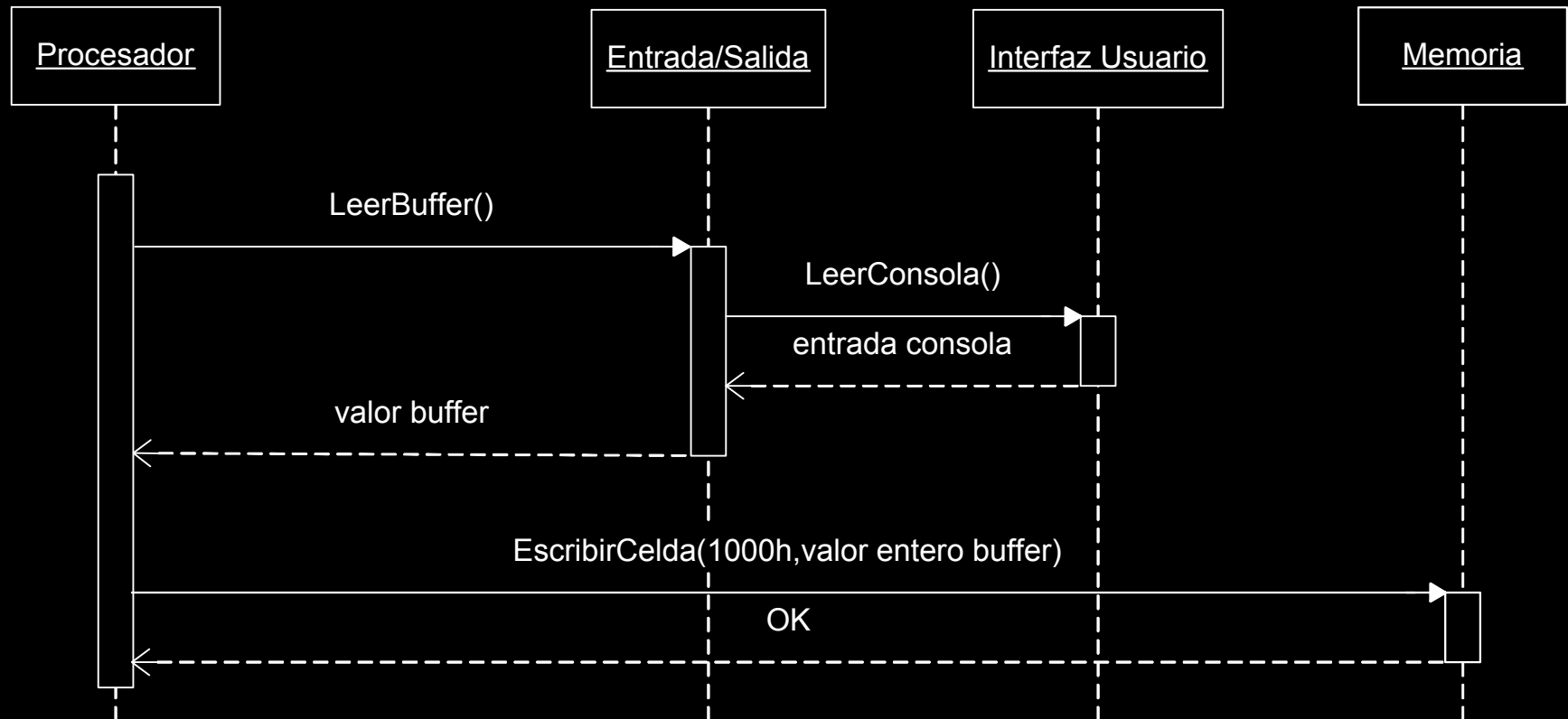
- Introducción
- Estado de la Cuestión
- **Solución**
 - Especificación de Requisitos
 - Ensamblador
 - **Simulador**
 - Interfaces
 - Pruebas
- Conclusiones
- Futuras Líneas de Trabajo

Modelo de objetos.

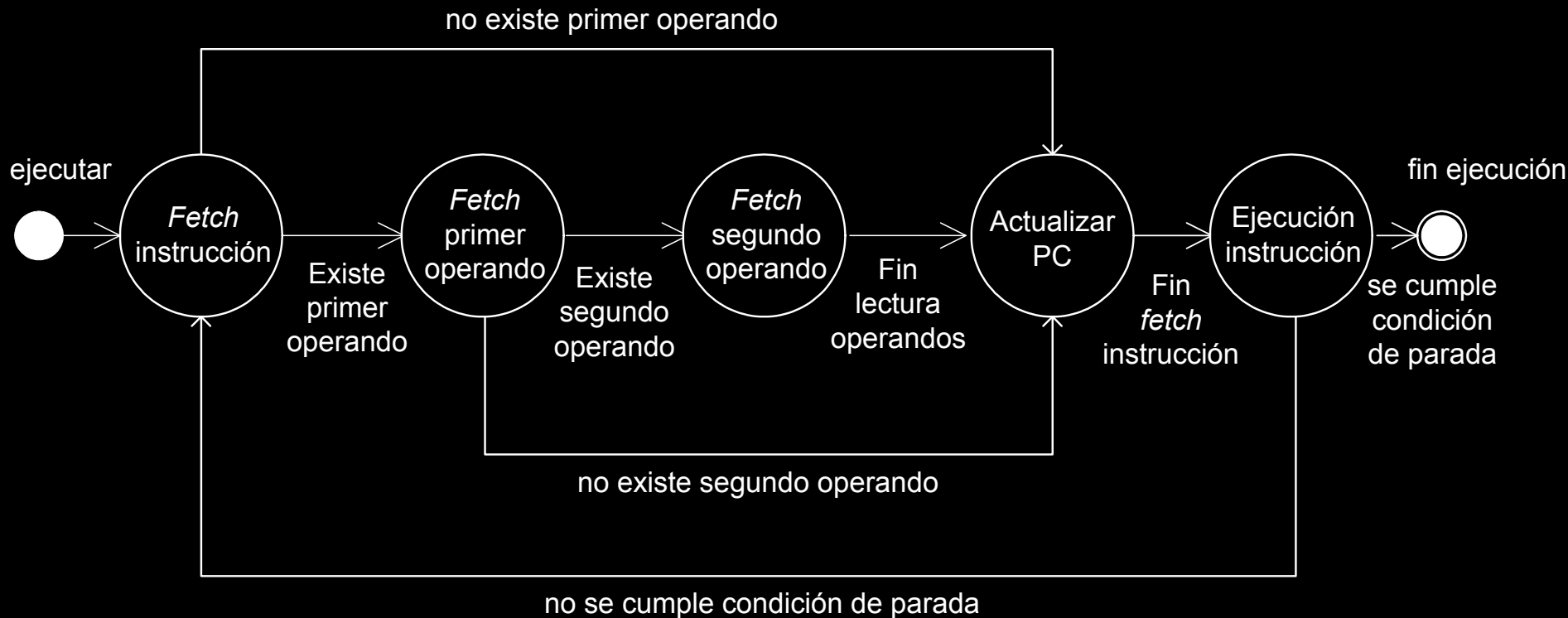


Modelo dinámico. Escenarios.

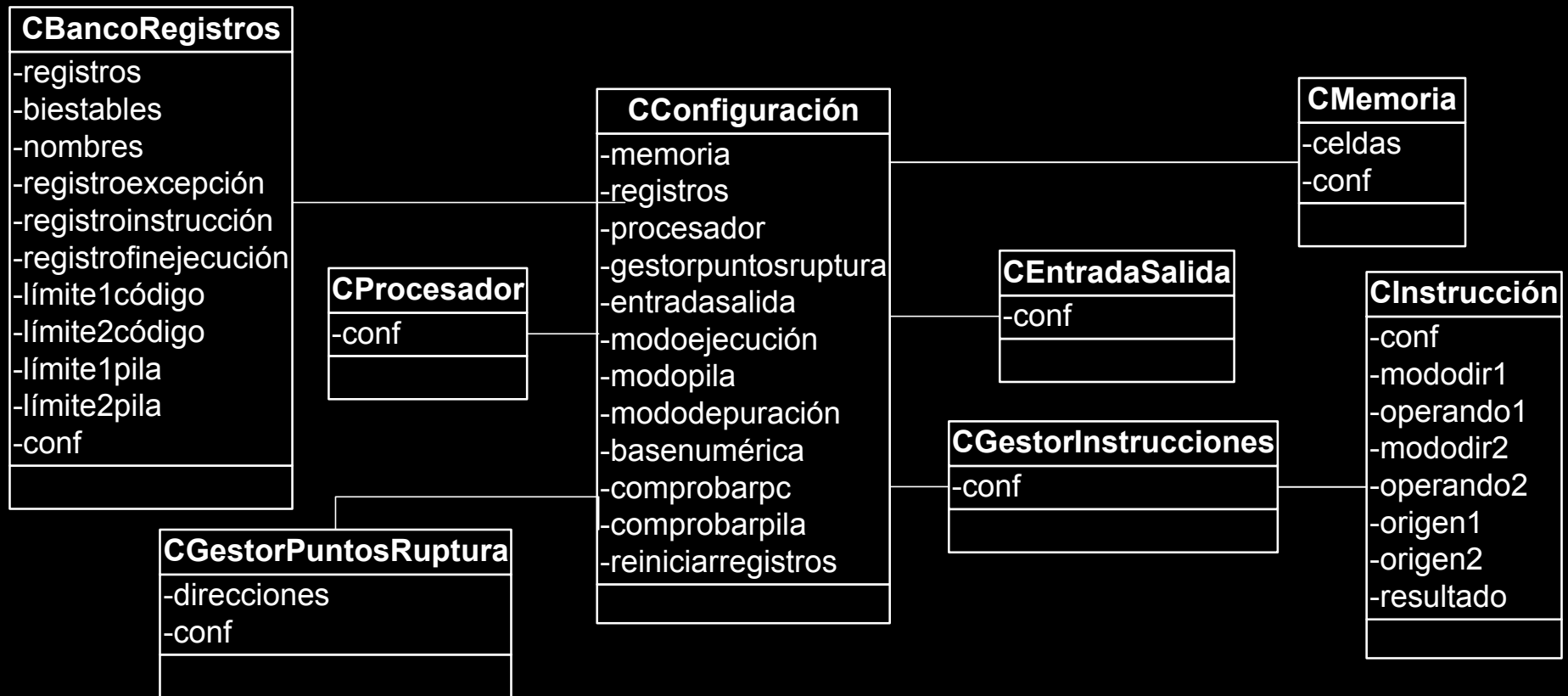
INCHAR /0x1000



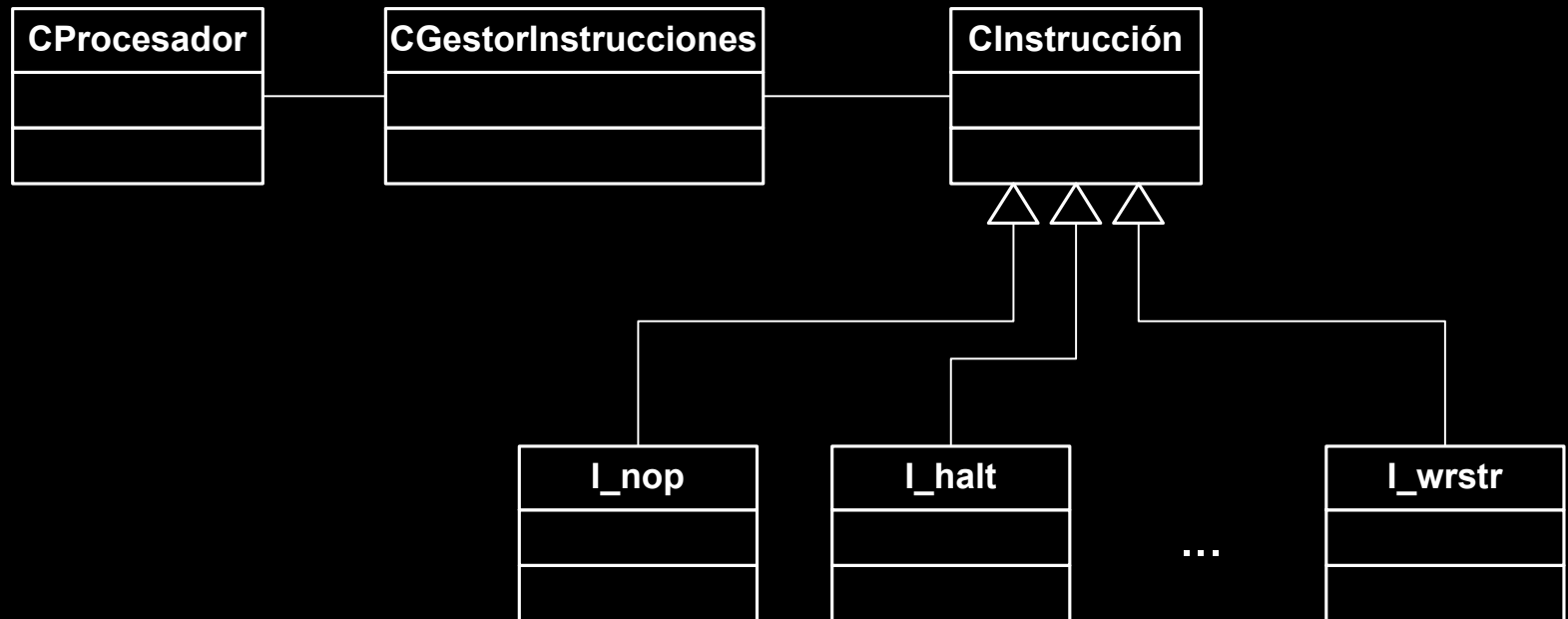
Modelo dinámico. Diagrama de estados.



Nuevo modelo de objetos.



Uso de la herencia. Clase **CInstrucción**.



SIMULADOR



Implementación.

- Código escrito en lenguaje C++.
- Herramientas de desarrollo:
 - *Borland C++ Builder 5 (DOS y Windows).*
 - *GNU Compiler Collection - gcc v.2.95.4 (Linux).*

SIMULADOR

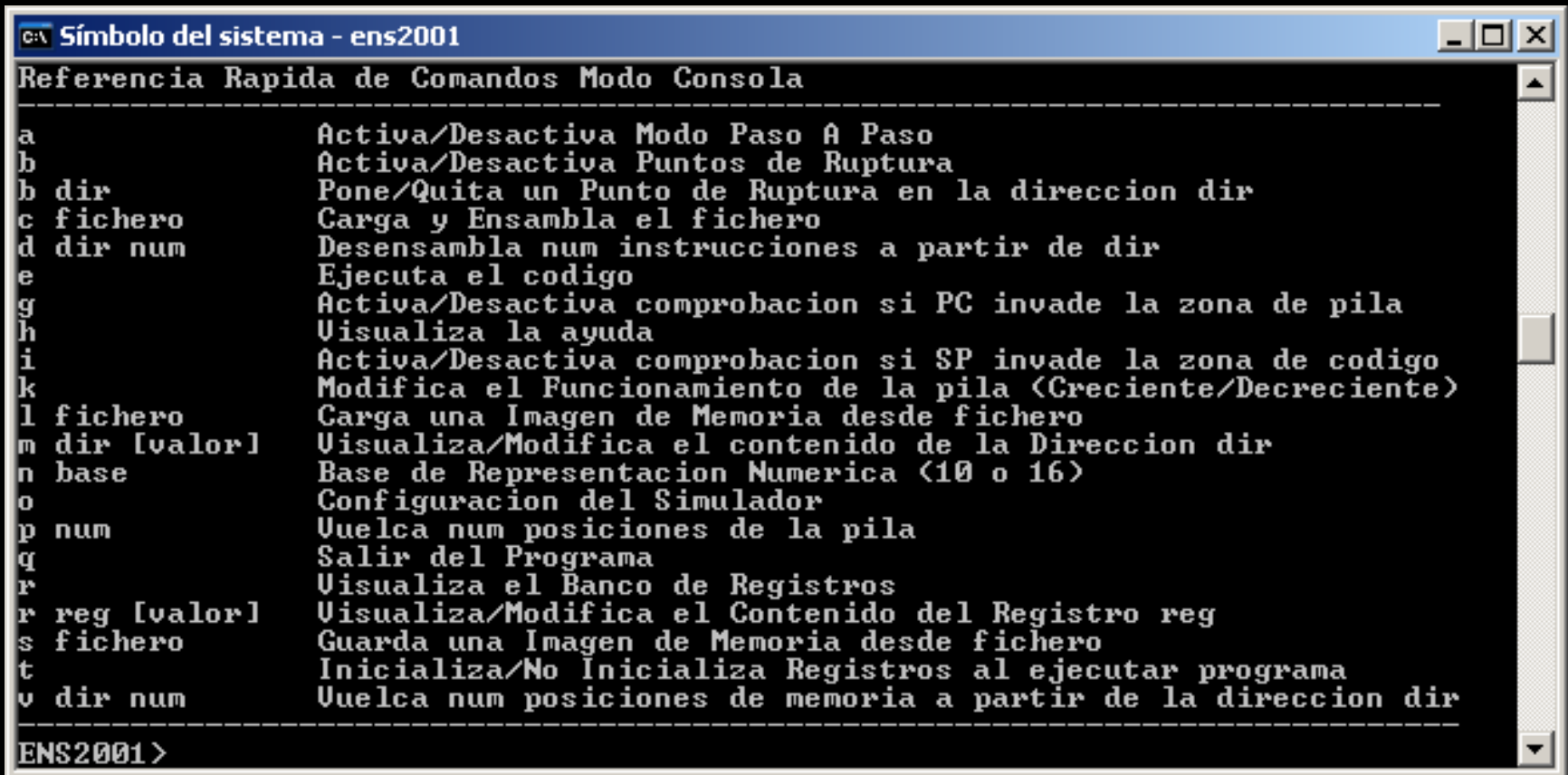
Sincronización entre simulador e interfaz.

- *DOS*. Uso de la función de biblioteca `kbhit` (no estándar). Dicha función no está disponible para *Linux*.
- *Windows*. Uso de hilos (*threads*).
 - La simulación se ejecuta en un hilo paralelo a la interfaz.
 - Durante la ejecución se desactiva la escritura en los componentes de la máquina virtual.

- Introducción
- Estado de la Cuestión
- **Solución**
 - Especificación de Requisitos
 - Ensamblador
 - Simulador
 - **Interfaces**
 - Pruebas
- Conclusiones
- Futuras Líneas de Trabajo

INTERFACES

Interfaz basada en línea de comandos.

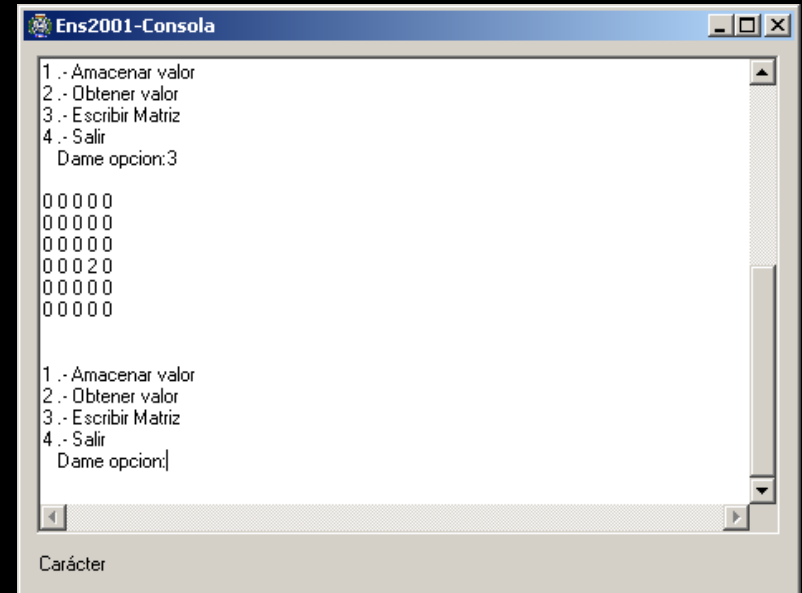
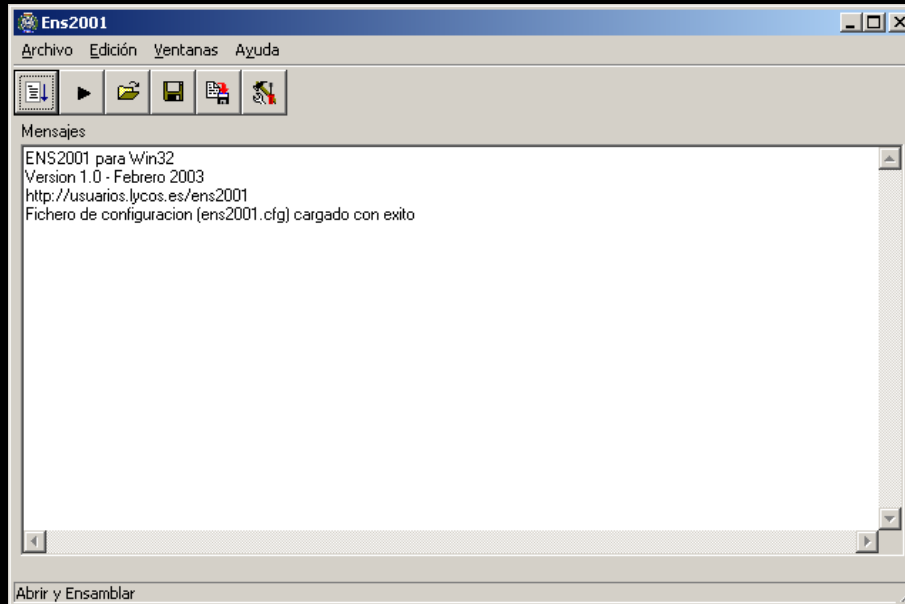


```
C:\> Símbolo del sistema - ens2001

Referencia Rapida de Comandos Modo Consola
-----
a          Activa/Desactiva Modo Paso A Paso
b          Activa/Desactiva Puntos de Ruptura
b dir      Pone/Quita un Punto de Ruptura en la direccion dir
c fichero  Carga y Ensambla el fichero
d dir num  Desensambla num instrucciones a partir de dir
e          Ejecuta el codigo
g          Activa/Desactiva comprobacion si PC invade la zona de pila
h          Visualiza la ayuda
i          Activa/Desactiva comprobacion si SP invade la zona de codigo
k          Modifica el Funcionamiento de la pila <Creciente/Decreciente>
l fichero  Carga una Imagen de Memoria desde fichero
m dir [valor] Visualiza/Modifica el contenido de la Direccion dir
n base     Base de Representacion Numerica <10 o 16>
o          Configuracion del Simulador
p num      Vuelca num posiciones de la pila
q          Salir del Programa
r          Visualiza el Banco de Registros
r reg [valor] Visualiza/Modifica el Contenido del Registro reg
s fichero  Guarda una Imagen de Memoria desde fichero
t          Inicializa/No Inicializa Registros al ejecutar programa
v dir num  Vuelca num posiciones de memoria a partir de la direccion dir
-----
ENS2001>
```

INTERFACES

Interfaz gráfica.



INTERFACES

Interfaz gráfica.

The image displays four graphical user interface windows for the Ens2001 system, each with a title bar and a close button (X).

Ens2001-Registros

Banco de Registros

A	0	R7	0
R0	134	R8	0
R1	0	R9	0
R2	0	PC	0
R3	0	SP	65535
R4	0	IX	0
R5	0	IY	0
R6	0	SR	0

Biestables

Z	0	P	0	C	0
S	0	V	0	H	0

Siguiente Instrucción
MOVE #32767,SP

Pila

Desde: 65535

Hasta: 65535

Código

Desde: 0

Hasta: 164

Reiniciar Valores

Ens2001-Memoria

Zona	Dirección	Valor
C	0	138
C	1	32767
C	2	14
C	3	2328
C	4	34
C	5	2064
C	6	256
C	7	208
C	8	256
C	9	1880
C	10	108

Reiniciar Valores

0 Ir A Dirección

Ens2001-Pila

Dirección	Valor
0	138
1	32767
2	14
3	1880
4	259
5	146
6	261
7	849
8	1280
9	4
10	1240

Ir a Puntero de Pila

0 Ir a Dirección

Ens2001-Fuente

Dirección	Instrucción
PC -> 0	MOVE #32767,SP
3	WRSTR /34
5	ININT .R1
7	PUSH .R1
9	CALL /108
11	POP .R2
13	CMP .R2,#0
16	BN /23
18	WRSTR /90
20	WRINT .R2
22	HALT

Desensamblar A Partir de PC

0 Desensamblar A Partir De

- Introducción
- Estado de la Cuestión
- **Solución**
 - Especificación de Requisitos
 - Ensamblador
 - Simulador
 - Interfaces
 - **Pruebas**
- Conclusiones
- Futuras Líneas de Trabajo

Pruebas Unitarias. (57 pruebas – 33 errores)

Estímulo	Se crean objetos válidos de la clase, por tanto, con valores iniciales comprendidos entre 0 y 65535. También se crean con el constructor por defecto y el constructor copia.
Respuesta	Se crean los objetos. Puede comprobarse el valor de sus atributos mediante los métodos de acceso.

Estímulo	Se crean objetos no válidos, es decir, con valores iniciales menores que 0 ó mayores que 65535.
Respuesta	Se lanza una excepción <code>runtime_error</code> .
Error	No se construyen bien los enteros cuando su valor es negativo dentro del rango permitido.
Solución	Se corrige el constructor <code>CEntero16b(const char *valor)</code> para que efectúe el tratamiento adecuado.

PRUEBAS

Pruebas de Integración. (14 pruebas – 14 errores)

Estímulo	Se introducen varias pseudoinstrucciones EQV , precedidas de etiquetas, para darles distintos valores válidos, comprendidos entre 0 y 65535.
Respuesta	Se comprueba que las etiquetas han tomado los valores indicados en el código fuente.

Estímulo	Para cada instrucción de salto condicional, ejecutarla dos veces, una en la que se cumpla la condición y otra en la que no.
Respuesta	Se comprueba que los saltos sólo se producen cuando la condición por la que se pregunta es cierta.
Error	La instrucción BNC se comporta igual que la instrucción BNV .
Solución	Se corrige la consulta al biestable de estado, que era errónea.

Pruebas de Sistema. (6 pruebas – 3 errores)

Programa `calculadora.ens`

- Permite probar todas las operaciones aritméticas y lógicas.
- También efectúa llamadas a subrutinas y entrada/salida por consola.

Error: La instrucción `MOD` no controlaba si el divisor era igual a cero.

Solución: Se corrige la acción semántica correspondiente a la regla `MOD`.

Beta-test y Pruebas de Regresión.

- Se han liberado 7 versiones *beta*.
- Ha permitido detectar y corregir 10 errores e introducir 5 mejoras.
- Tras la corrección de los fallos, se deben efectuar pruebas de regresión.
- Datos de uso durante la fase *beta*:

	<i>Windows</i>	<i>DOS</i>	<i>Linux</i>	Versiones	% uso
Abril-2002 a Mayo-2002	20	0	N/A	<i>beta 1 a beta 6</i>	51%
Junio-2002 a Septiembre-2002	33	7	0	<i>beta 6</i>	59%
Octubre-2002 a Febrero-2003	9	2	1	<i>beta 7 → 1.0</i>	70%

CONCLUSIONES



- Introducción
- Estado de la Cuestión
- Solución
 - Especificación de Requisitos
 - Ensamblador
 - Simulador
 - Interfaces
 - Pruebas
- Conclusiones
- Futuras Líneas de Trabajo

CONCLUSIONES

- Consecución de una herramienta de gran valor didáctico.
- Proceso de desarrollo muy completo.
- Se han cumplido las expectativas técnicas
 - Interfaz independiente del motor.
 - Desarrollo multiplataforma (*DOS, Windows, Linux*)
- Satisfacción de los usuarios.
- Facilidad de mantenimiento.

CONCLUSIONES

Dificultades resueltas durante el desarrollo.

- Informar los errores en el código fuente del usuario de forma específica, localizada, completa y legible.
- Combinar código generado en *C* (por *Flex* y *Bison*) y *C++*.
- Implementación de los *threads*.

FUTURAS LÍNEAS DE TRABAJO



- Introducción
- Estado de la Cuestión
- Solución
 - Especificación de Requisitos
 - Ensamblador
 - Simulador
 - Interfaces
 - Pruebas
- Conclusiones
- Futuras Líneas de Trabajo

FUTURAS LÍNEAS DE TRABAJO

Mejoras sobre el panorama actual.

- Ensamblador.
 - Abandonar definitivamente la interfaz textual.
 - Implementar una versión gráfica para *X-Windows*.
 - Portar la herramienta a un entorno independiente de la arquitectura sobre la que se ejecuta.
 - Incorporación de un editor de texto con resaltado de sintaxis (*syntax highlighting*).

FUTURAS LÍNEAS DE TRABAJO

Mejoras sobre el panorama actual.

- Simulador.
 - Completar los modos de direccionamiento con aquéllos que se consideraron poco usuales.
 - Mayor soporte a las excepciones.
 - Modificar la configuración del simulador mediante directivas en el código fuente.
 - Soporte para números reales (*IEEE 754*).

FUTURAS LÍNEAS DE TRABAJO

El futuro.

- Simulador genérico de la arquitectura *Von Neumann*.
 - Características parametrizables (memoria, registros, UAL, juego de instrucciones).
 - Unidad de control microprogramada.
 - Simulación de periféricos. Interrupciones. DMA. Excepciones.
 - Representación gráfica de los componentes simulados.



ENS2001

Ensamblador y Simulador del Lenguaje Ensamblador *IEEE 694*

Autor: Federico J. Álvarez Valero

Tutor: José Luis Fuertes Castro

Facultad de Informática – Marzo 2003